Independent Researcher MIST.cash—FOCBB, Shhtarknet shramee@proton.me

Probabilistic Pairing Proofs

Shramee Srivastav10009-0009-7208-3478

October 24, 2025

Abstract

In this paper, we introduce a public-coin interactive proof system for efficiently proving the correctness of elliptic curve pairing computations. Elliptic curve pairings form the mathematical foundation of many modern cryptographic protocols, yet their verification remains computationally intensive.

In resource-constrained environments—including BitcoinVM, blockchain smart contracts, ZKVMs, and recursive zero-knowledge proof systems—traditional pairing verification remains expensive. This limits the applicability and affordability of privacy-preserving applications, effectively hindering widespread adoption of blockchain technology.

We systematically present a progression of state-of-the-art techniques: El Housni's foundational R1CS pairing optimizations [**?**], Feltroidprime's enhanced extension field arithmetic [**?**], and Novakovic and Eagen's final exponentiation elimination [**?**].

Building upon these foundations, our approach consolidates Miller loop iterations and final exponentiation verification into univariate polynomial relations verified through efficient polynomial identity testing. For BN254 3-pairing verification (standard in Groth16), our Cairo implementation achieves:

- **45% reduction** in Miller loop field operations
- **77% reduction** in commitment overhead (Fiat-Shamir hashing)
- **37% overall** speedup for complete pairing verification

Elliptic curve pairings Zero-knowledge proofs Polynomial identity testing Blockchain verification.

Contents

List of Algorithms

List of Tables

1 Introduction

Blockchain technology is ready for widespread adoption—we have reached a scale where processing Visa-like volumes on-chain is within reach. However, privacy remains a significant barrier to entry for many users and applications. Zero-knowledge proofs (ZKPs) have emerged as a powerful solution to this challenge.

1.1 Motivation

Elliptic curve pairings represent fundamental building blocks for numerous cryptographic systems including Groth16 [**?**], PLONK [**?**], BLS signature schemes [**?**], and KZG polynomial commitment schemes [**?**].

However, **pairings are computationally intensive**. In execution environments such as blockchain virtual machines, recursive proof systems, and arithmetized circuits, the costs associated with this computation become prohibitive for practical deployment.

1.2 Our Contribution

We present a novel optimization for pairing computation through a public-coin interactive proof system that consolidates the pairing operations into unified polynomial verifications under the Schwartz-Zippel lemma. By treating complete loop iterations as single polynomial relationships rather than sequences of individual field operations [**?**], our approach achieves:

- **50% reduction** in the pairing computation complexity
- **Over 70% decrease** in auxiliary data overhead
- Practical deployment enabling ZK proof verification within blockchain transaction limits

Our key insight is that instead of optimizing individual field operations, we can treat complete Miller loop iterations as single polynomial relationships, dramatically reducing both computational and communication costs.

Implementation and Impact: Garaga [**?**], a state-of-the-art elliptic curve tooling library for Cairo and Starknet, implements our two round interactive proof system with polynomial batching. These techniques allowed Garaga to fit their pairing transaction within Starknet's 4000 calldata limit while simultaneously minimising the gas costs.

1.3 Paper organization

The remainder of this paper is organized as follows.

Section 2 provides necessary background on pairings and constraint systems.
Section 3 surveys the evolution of optimization techniques that form our foundation.
Section 4 presents our unified Miller loop optimization framework.

Section 5 outlines the public-coin interactive proof systems and algorithms for prover and verifier.

Section 6 concludes with implications and future work.

2 Background

Algorithm 1 Optimal Ate pairing over Barreto–Naehrig curves [**?**]

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$

Output: $a_{opt}(Q, P)$

Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
$T \leftarrow Q, f \leftarrow 1$
for $i = L - 2$ to 0 **do**
    $f \leftarrow f^2 \cdot l_{T,T}(P)$
    $T \leftarrow [2]T$
    if $s_i = 1$ **then**
        $f \leftarrow f \cdot l_{T,Q}(P)$
        $T \leftarrow T + Q$
    end if
    if $s_i = -1$ **then**
        $f \leftarrow f \cdot l_{T,-Q}(P)$
        $T \leftarrow T - Q$
    end if
end for
$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$f \leftarrow f \cdot l_{T,Q_1}(P), T \leftarrow T + Q_1$
$f \leftarrow f \cdot l_{T,-Q_2}(P), T \leftarrow T - Q_2$
$f \leftarrow f^{p^{12}-1/r}$
return f

2.1 Pairing Computation

An elliptic curve pairing is a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are groups of points on elliptic curves over finite fields, and $\mathbb{G}_T$ is a multiplicative group in an extension field. For practical cryptographic applications, we require that this map be efficiently computable.

The computation of an optimal ate pairing—the focus of our implementation—consists of two primary phases:

1. **Miller loop**: The core iterative computation that evaluates line functions at pairs of points from $\mathbb{G}_1$ and $\mathbb{G}_2$, accumulating intermediate results to produce a value in $\mathbb{F}_{q^{12}}$.

2. **Final Exponentiation**: This step raises the Miller loop output to the power $(q^{12} - 1)/r$, where r is the order of the groups, ensuring the result lies in the correct subgroup $\mathbb{G}_T$.

While both phases present optimization opportunities, the constraint-based verification of these computations introduces unique challenges and possibilities that differ significantly from native implementations.

2.2 Schwartz–Zippel lemma

The Schwartz–Zippel lemma (or more precisely, the Demillo–Lipton–Schwartz–Zippel lemma [?, ?, ?, ?]) provides a probabilistic method for testing polynomial identities. The lemma states that for a non-zero polynomial $P(x_1, x_2, \ldots, x_n)$ of total degree d over a finite field $\mathbb{F}$, when evaluated at uniformly random field elements, the probability that P evaluates to zero is at most $d/|\mathbb{F}|$.

This probabilistic bound becomes negligible for cryptographically sized fields where $d \ll |\mathbb{F}|$, making it practical to verify polynomial identities by evaluation instead of coefficient-wise operations.

2.3 Constraint-Based Computation Models

In the context of arithmetization protocols and virtual machines (such as Starkware's Cairo VM [?], R1CS [?], Plonkish [?], etc.), where field operations are expressed as algebraic constraints, the computational landscape differs dramatically from native implementations. Operations that are traditionally expensive can become remarkably efficient. Field inversions, for example, can be computed at a fraction of their usual cost, often reducing their relative costs from over 100× to about the same cost as multiplication. Conversely, seemingly straightforward operations may require complex constraint representations.

Recognizing and exploiting these unique computational properties is essential for unlocking hidden performance potential. Youssef El Housni's pioneering work **Pairings in Rank-1 Constraint Systems** [?] represents, to our knowledge, the first systematic exploration of these optimization opportunities, providing a breakthrough in the practicality of recursive proving by thoughtfully leveraging the distinctive cost model inherent to constraint-based systems.

3 Related Work

The landscape of efficient pairing verification in constraint systems has evolved through several key contributions. El Housni's **Pairings in Rank-1 Constraint Systems** [?] established the theoretical framework for optimizing pairings in Rank-1 Constraint Systems, introducing techniques for efficient Miller loop implementation and final exponentiation that exploit the unique properties of constraint-based computation models. This work demonstrated that operations traditionally considered expensive in native implementations could be restructured to achieve remarkable efficiency gains in arithmetized environments.

Feltroidprime's subsequent research on **enhanced extension field multiplication techniques** [?] introduced polynomial verification methods using the Schwartz-Zippel lemma for $\mathbb{F}_{q^{12}}$ arithmetic operations. By representing extension field elements as polynomials and verifying their operations through polynomial arithmetic, this approach achieved significant performance improvements in the pairing computations, particularly for emulated pairing circuits where traditional field operations are costly.

Novakovic and Eagen's work **On Proving Pairings** [?] presented breakthrough optimization strategies that enable the elimination of final exponentiation overhead through residue witness techniques. Their approach demonstrates that two elements in $\mathbb{F}_{q^{12}}$ can be proven equivalent without expensive exponentiation by introducing witness elements that satisfy specific algebraic relationships, effectively embedding the final exponentiation verification into the Miller loop computation itself.

3.1 Pairings in R1CS – Housni 2022

El Housni's seminal work [?] represents the first systematic exploration of pairing optimization within constraint-based computation models, providing techniques that extend beyond R1CS to benefit any arithmetized environment, including Starkware's Cairo VM [?].

3.1.1 Cost of inversions

Housni notes that the cost of inversions is much lower in constraint systems, almost the same as multiplication. Over $\mathbb{F}_p$, inversions cost 2 constraints: one for computing $x \cdot y$ where y is provided from the hint, and one equality check $x \cdot y \stackrel{?}{=} 1$. Division can be similarly reduced to just two constraints. For $c = a/b$ where c is the input from hint, we compute $b \cdot c$ and equality check $b \cdot c \stackrel{?}{=} a$. This is further generalised to field extensions.

3.1.2 Affine coordinates

Since division cost about as much as multiplications, we can represent points in affine coordinates for lower operation counts. Point doubling and addition formulas are provided in section 6.1 of [?]. Doubling costs 12C and addition costs 10C.

3.1.3 Avoid doubling

For double and add operations on non zero bits of the Miller loop, instead of computing [2]S + Q, computing S + Q + S saves 2C per non zero bit. Taking this a step further, computation of y coordinate for the intermediate $S + Q$ point can be ommitted and λ_1 can be directly used in computation of λ_2. This **costs 17C**, that's a **23% savings** over 22C for [2]S + Q.

3.1.4 Line evaluations

Following [?], the lines are sparse elements in $\mathbb{F}_{q^{12}}$ of the form $ay_P + bx_P \cdot w + c \cdot w^3$ with $a, b, c \in \mathbb{F}_{p^2}$. The multiplication of these sparse elements costs 39C. Housni proposes representing the lines in the form $1 + \frac{b'x_P}{y_P} \cdot w + \frac{c'}{y_P} \cdot w^3$ which makes it sparser (notice unit first coefficient resulting in free multiplication).

Terms $\frac{x_P}{y_P}$ and $\frac{1}{y_P}$ can be precomputed at the cost of 2C, and reused throughout the miller loop. Untwisting isomorphisms for this form are given on pages 15–16 in section 6.1 of [?]. This representation only **costs 30C**, that's a **30% savings** over 39C for the original representation.

3.1.5 Performance

Table ?? summarizes the arithmetic costs for $\mathbb{F}_{q^{12}}$ as presented in Housni's paper.

Table 1: Arithmetic in Housni's paper [?]

	Mul	Square	Div	Sparse Mul
$\mathbb{F}_{q^{12}}$	54C	36C	66C	30C

3.2 Faster Extension Field Multiplications – Feltroidprime 2023

Feltroidprime [?] showcases a novel approach to extension field multiplications that exploits a counterintuitive property of circuit-based computation: operations traditionally viewed as expensive (such as polynomial divisions) can become highly efficient in constraint systems. Moving away from the conventional towered extension approach, he proposes returning to direct extensions of $\mathbb{F}_p$ to achieve significant performance improvements.

3.2.1 Isomorphisms among towered to direct extensions

Feltroidprime walks through the construction of mapping functions to go back and forth between direct and towered extensions.

He starts with the tower for BN254 (as used in gnark [?]) and obtains irreducible polynomial – $P_{12}(x) = x^{12} - 18x^6 + 82$

And prepares isomorphisms between tower and direct representations [?] which we will not include in this paper for brevity.

3.2.2 Multiplication and Schwartz–Zippel lemma

Feltroidprime proposes using Euclidean division of polynomials to multiply $A, B \in \mathbb{F}_{q^{12}}$. The computation happens as follows:

1. Treat A, B as polynomials of degree less than 11, with coefficients in $\mathbb{F}_p$.
2. Compute the product $C = A \cdot B$ as a polynomial of degree less than 22 for $\mathbb{F}_{q^{12}}$.
3. Perform Euclidean division of C by the irreducible polynomial $P_{12}(x)$ to obtain the quotient Q and remainder R: $\boxed{A(x) \cdot B(x) = Q(x) \cdot P_{12}(x) + R(x)}$
4. $R \in \mathbb{F}_{q^{12}}$ is used as the result of the multiplication between A and B.

Inside the circuit, this computation can be provided as a hint and verified with the Schwartz–Zippel lemma. The Fiat–Shamir heuristic can be used to generate the challenge with the transcript.

3.2.3 Deferred Evaluation and Batching

In Section 4, Feltroidprime discusses deferring the evaluation to the end of the algorithm and combining it with a polynomial commitment scheme to batch the verifications. This batching transforms hundreds of individual verifications into a single equation. So they can be checked at a single random point, amortizing the costs for Fiat-Shamir hashing.

The batching process is a random linear combination of all the equations to be verified. For n multiplications, we have:

$$\sum_{i=0}^{n-1} c_i * A_i(z) * B_i(z) = \sum_{i=0}^{n-1} c_i * R_i(x) + \sum_{i=0}^{n-1} c_i * Q_i(z) * P_{12}(z) \tag{1}$$

3.2.4 Performance

For performance analysis, we will use the batched verification version.

The implementation receives the computed RHS polynomial aggregation $Q_{Agg} = \sum_{i=0}^{n-1} c_i * Q_i$ as hint. The LHS aggregation, $\sum_{i=0}^{n-1} c_i * A_i(z) * B_i(z) - R_i(x)$ is computed within the verifier circuit. While the amortized cost computing the $Q_{Agg}(z)$ and $P_{12}(z)$ once in the circuit approaches zero as the number of batched operations increases, we conservatively account for 1C per multiplication to provide fair cost comparison.

Multiplication costs 12C each for A, B and R evaluation, 1C for multiplying $A(z)$ and $B(z)$ evaluations, plus 1C for preparing the RLC c_i multiplication and conservative 1C for RHS aggregation.

For **squarings**, the evaluation of $A(z)$ is just used again, costing 12C less than multiplication.

Division costs multiplication plus additional 12C for assertion.

Sparse element multiplication costs Evaluations for A, B and R where B contains 5 non-zero coefficients, one out of which is unitary free multiplication. The costs comes at 31C which is 4C more than squaring which has free second polynomial because $A = B$ for squarings.

Table 2: Arithmetic in Feltroidprime's paper [**?**]

	Mul	Square	Div	Sparse Mul
$\mathbb{F}_{q^{12}}$	39C*	27C*	51C*	31C*
*Costs exclude Fiat-Shamir commitment overhead				

3.3 Eliminating Final Exponentiation – Novakovic & Eagen 2024

The mathematical structure underlying pairing based cryptography involves equivalence classes. Tate pairings (as described in [**?**] and [**?**]) result in the output in the quotient group $F_{q^k}/(F_{q^k})^r$. This requires the output of the Miller loop to be raised to a power of $(q^k - 1)/r$. This step, called **the final exponentiation**, normalizes the Miller loop output by clearing the r-th residue part. This normalization ensures equivalent pairing outputs produce identical results.

This operation is particularly expensive in extension fields like $\mathbb{F}_{q^{12}}$, because $(q^{12}-1)/r$ becomes very large.

3.3.1 Equivalence class

Novakovic and Eagen [**?**] highlight that the final exponentiation can be eliminated when proving pairings. The expensive exponentiation can be replaced with an efficient residue

check. The key insight is that instead of normalizing pairing outputs through final exponentiation, we can directly verify equivalence using a residue–witness based approach.

Equivalence class representation: Two elements $A, B \in \mathbb{F}_{q^{12}}$ are equivalent in the pairing context if there exists some witness c such that $A \cdot c^r = B$.

$$A \equiv B \iff \exists c \in \mathbb{F}_{q^{12}} : A \cdot c^r = B$$

3.3.2 Equivalence via Residue Check

So we have,

$$\begin{aligned} A \cdot c^r &\equiv A \\ \text{After final exponentiation,} & \\ (A \cdot c^r)^{(q^{12}-1)/r} &\equiv A^{(q^k-1)/r} \\ A^{(q^k-1)/r} \cdot c^{r*(q^k-1)/r} &\equiv A^{(q^k-1)/r} \\ A^{(q^k-1)/r} \cdot c^{q^k-1} &\equiv A^{(q^k-1)/r} \\ \text{With Fermat's Little Theorem,} & \\ A^{(q^k-1)/r} \cdot 1 &\equiv A^{(q^k-1)/r} \end{aligned}$$

If we expect the result of the pairing, $A \in \mathbb{F}_{q^k}$, to be equal to some desired output $B \in \mathbb{F}_{q^k}$, we can use the equivalence relation to establish there exists some $c \in \mathbb{F}_{q^k}$ such that $A \cdot c^r = B$.

This can be generalized to $A \cdot w^{rt} = B$ for $c = w^t$ in $A \cdot c^r = B$.

3.3.3 Hasse bound and exponentiation by r

According to Hasse theorem [**?**], $q + 1 - r \leq 2\sqrt{q}$ for an elliptic curve over finite field $\mathbb{F}_q$ and r the order of the curve.

The exponentiation by r can be written as exponentiation by $q+1-t$. Where $t \leq 2\sqrt{q}$ is much smaller exponentiation in worst case. And the computation of c^q is efficiently computed by exploiting the Frobenius endomorphisms.

3.3.4 Parameters for BN254 curve

Section 4.2 and 4.3 of [**?**] dive into finding optimal parameters for the curve to establish $A \cdot c^\lambda = B$

For Ethereum's BN254 curve, the optimal parameter is $\lambda = 6x+2+q-q^2+q^3$, which decomposes into two efficiently computable components:

1. **Miller loop component** $(6x+2)$ — Integrated into existing Miller loop operations
2. **Frobenius component** $(q - q^2 + q^3)$ — Computed using efficient Frobenius mappings

This eliminates the expensive final exponentiation, replacing it with a few additional Miller loop operations and three Frobenius mappings.

3.3.5 Performance

Replacing the final exponentiation by residue witness check saves over 85% of final exponentiation costs for BN254 curve, reducing the constraints required for whole pairing by 32%.

Table **??** summarizes the performance impact of eliminating the final exponentiation step, comparing the original pairing costs with those after applying the residue witness technique.

Table 3: Performance impact of residue witness check [**?**]

Computation	**Full Pairing**	**Witness Check [?]**	**Change**
Emulated BN254 pairing			
Final Exponentiation	295,823C	38,276C	-87.1%
Total cost	784,050C	526,503C	**-32.8%**
Native BLS12377 [?] pairing			
Final Exponentiation	6,130C	396C	-93.5%
Total cost	14,536C	8,802C	**-39.4%**

4 Pairing Operations as Polynomials

The techniques discussed in the previous section are outlined in Algorithm **??**. This algorithm combines El Housni's R1CS optimizations [**?**] with Novakovic and Eagen's residue witness technique for final exponentiation elimination [**?**].

We now turn our attention to our primary contribution. Building upon Feltroidprime's direct extension multiplication framework, we present a novel optimization that significantly improves efficiency across all phases of the pairing computation. Our approach consolidates operations across the entire process into unified polynomial verifications. This includes the Miller loop step operations as well as the final exponentiation (via residue witness) verification. Each phase is expressed as a polynomial relationship and verified using the Schwartz-Zippel lemma, substantially reducing constraint count while maintaining cryptographic security.

4.1 Strategic Polynomial Segmentation

A critical consideration in our design is managing polynomial degree growth. While consolidating operations into fewer polynomials reduces overhead, we must balance this against polynomial degree constraints to maintain verification efficiency.

4.1.1 Squaring operations and polynomial degree

Squaring operations **double the polynomial degree**. In traditional $\mathbb{F}_{q^{12}}$ arithmetic, squaring operations are typically cheaper than general multiplications. However, in our polynomial verification framework, squaring the accumulator F doubles the polynomial degree, making placement critical. When positioned after other operations, squaring becomes the most degree-expensive operation in the sequence.

Algorithm 2 Optimal Ate pairing with Residue witness check

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$
Internal $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: $a_{opt}(Q, P)$

Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

for $i = L - 2$ to 0 **do**
  $f \leftarrow f^2$
  $f \leftarrow f \cdot l_{T,T}(P)$ ▷ Repeat for T_i, P_i
  $T \leftarrow [2]T$ ▷ Repeat for T_i
  if $s_i = 1$ **then**
    $f \leftarrow f \cdot c^{-1}$ ▷ Residue witness
    $f \leftarrow f \cdot l_{T,Q}(P)$ ▷ Repeat for T_i, Q_i, P_i
    $T \leftarrow T + Q$ ▷ Repeat for T_i, Q_i
  else if $s_i = -1$ **then**
    $f \leftarrow f \cdot c$ ▷ Residue witness
    $f \leftarrow f \cdot l_{T,-Q}(P)$ ▷ Repeat for $T_i, -Q_i, P_i$
    $T \leftarrow T - Q$ ▷ Repeat for $T_i, -Q_i$
  end if
end for

$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ ▷ Repeat for Q_{1i}, T_i
$f \leftarrow f \cdot l_{T,Q_1}(P)$ ▷ Repeat for Q_{1i}, T_i, P_i
$T \leftarrow T + Q_1$ ▷ Repeat for Q_{1i}, T_i
$f \leftarrow f \cdot l_{T,-Q_2(Q)}(P)$ ▷ Repeat for Q_i, T_i, P_i
▷ Skip $T \leftarrow T - Q_2(Q)$

if $w = 1$ **then**
  $f \leftarrow f \cdot W$
else if $w = 2$ **then**
  $f \leftarrow f \cdot W^2$
end if
$f \leftarrow f \cdot c^{-q} \cdot c^{q^2} \cdot c^{-q^3}$ ▷ Uses Frobenius maps, Negative exponents use c^{-1}

return f

4.1.2 Natural segmentation points

The optimal points to segment our unified polynomial are immediately *before* squaring the accumulator. By making squaring the first operation in each polynomial segment, we:

1. Minimize the degree growth impact—the squaring happens on a fresh polynomial of degree 11
2. Accumulate all subsequent multiplications at manageable degree increases
3. Maintain relatively uniform polynomial degrees across segments

This observation naturally leads to segmenting operations at each Miller loop step boundary, where the accumulator F is squared. The remaining operations are then segmented to match the degree profile of Miller loop step polynomials, ensuring balanced verification costs across all phases.

4.2 Unified Polynomial Framework

Any sequence of field operations in $\mathbb{F}_{q^{12}}$[1] can be expressed as a single polynomial equation of the form:

$$A_1(x) \cdot \ldots \cdot A_n(x) = Q(x) \cdot P_{12}(x) + R(x) \text{ where } A_1, \ldots, A_n, R \in \mathbb{F}_{q^{12}}$$

This unified framework allows us to batch verify entire computational phases rather than individual operations, achieving dramatic improvements in both constraint counts and commitment overhead across the complete pairing computation.

4.3 Comparative Analysis

The Miller loop is comprised of steps for each individual bit, which in NAF (non-adjacent form) [**?**], may be +1, -1 or 0. Instead of performing a polynomial commitment for each individual extension field arithmetic operation separately, we propose the whole bit operation be viewed as one polynomial. And commitment be made to that polynomial.

To get a closer look into effectiveness of our scheme, we will make a real world application comparison for verification of three–pairing Miller loop which is common for verification of Groth16 [**?**]. We benchmark our approach against the fastest known polynomial commitment adaptation [**?**].

4.3.1 Previous scheme

Adapting faster extension field multiplication [**?**] idea to line **??** and **??** of Miller loop algorithm **??** for a 3–pair Miller loop, we have following polynomial relationships to assert.

[1]Elements need not have all 12 non-zero coefficients, but should align with the $\mathbb{F}_{q^{12}}$ representation (e.g., sparse line functions).

$$F(x) \cdot F(x) \stackrel{?}{=} Q_1(x) \cdot P_{12}(x) + T_1(x)$$
$$T_1(x) \cdot L_1(x) \stackrel{?}{=} Q_2(x) \cdot P_{12}(x) + T_2(x)$$
$$T_2(x) \cdot L_2(x) \stackrel{?}{=} Q_3(x) \cdot P_{12}(x) + T_3(x)$$
$$T_3(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

F, T_1, T_2, T_3, R cost 12C each for evaluation, 4C for $\mathbb{F}_q$ multiplication, and L_1, L_2, L_3 require 4C each for evaluation. All the remainder hints need to be provided for commitment, so this is 12 calldata commitment for R and intermediate remainder polynomials T_1, T_2, T_3. **Costs sum up to 76C** ($5 * 12C + 3 * 4C + 4C$) **and 48 commitments** in $\mathbb{F}_q$ $(4 * 12)$. Resulting remainder polynomial $R \in \mathbb{F}_{q^{12}}$ is the result of the Miller loop step.

4.3.2 Our scheme

In our approach, instead of working with individual arithmetic operations, we treat the entire bit operation as a single polynomial. In our scheme line **??** and **??** of 3–pair Miller loop (Algorithm **??**) looks like this.

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

Similar to the previous approach, F and R cost 12C each for evaluation, 4C for $\mathbb{F}_q$ multiplication, and L_1, L_2, L_3 require 4C each for evaluation, but we eliminate intermediate (T_* polynomials above) commitments and evaluations. Only the remainder hint needs commitment, so this is 12 calldata commitment for R. **Costs sum up to 40C** ($2 * 12C + 3 * 4C + 4C$) **and 12 $\mathbb{F}_q$ commitments**.

This results in **47% fewer constraints** and a whopping **75% savings in the commitments**. When implemented on a blockchains for verfication of ZK proofs, commitment savings become far more critical because of limitations and costs associated with calldata needed for hints.

4.3.3 Performance Comparison

Table **??** compares the computational costs and commitment overhead between the schemes for a single 0-bit Miller loop operation with 3 pairs.

4.4 Operation Polynomials

Our implementation uses different polynomials for different bits in the Miller loop. Let' walk through a 3-pairing setup which Inversions used widely for Groth16 verification. We will describe the polynomials for zero bits, non-zero bits and correction step.

4.4.1 Miller loop: Zero Bit Equation

As seen in **??** for zero bit operations (line **??** and **??** with three $l_{T,T}(P)$ lines in Algorithm **??**), the verification equation is:

Table 4: 3-Pair Miller loop 0-bit constraints comparison

Scheme	Constraints	$\mathbb{F}_q$ Commitments	Improvement
[?]	132C	—	Baseline
[?]	76C*	48	42%
Our scheme	**40C***	**12**	**70%**

[?] (Table **??**): 1 square (36C) + 3 sparse mul (3×30C) = 132C
[?]: 5 evals (5×12C) + 3 line evals (3×4C) + 4 muls (4×1C) = 76C
Our scheme: 2 evals (2×12C) + 3 line evals (3×4C) + 4 muls (4×1C) = 40C
*Costs exclude Fiat-Shamir commitment overhead

$$F^2(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) = R(x) + Q(x) \cdot P_{12}(x) \tag{2}$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_1, L_2, L_3 \in \text{Sparse}_{01379}$	The line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	36C Evals, 4C Muls and 1C RLC		**41**
Q *(Amortized See* **??***)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	38	39

Table 5: Zero bit operation polynomial components

4.4.2 Miller loop: Non-Zero Bit Equation

For non-zero bit operations(lines **??**, **??**, **??**, **??** with three pairs of lines in Algorithm **??**), the Miller loop performs both doubling and addition steps. So for each pair we have two Sparse_{01379} lines. The verification equation becomes:

$$F^2(x) \cdot W(x) \cdot L_{\text{d1}}(x) \cdot L_{\text{d2}}(x) \cdot L_{\text{d3}}(x) \cdot L_{\text{a1}}(x) \cdot L_{\text{a2}}(x) \cdot L_{\text{a3}}(x) = R(x) + Q(x) \cdot P_{12}(x) \tag{3}$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_{\text{d1}}, L_{\text{d2}}, L_{\text{d3}} \in \text{Sparse}_{01379}$	Doubling line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{\text{a1}}, L_{\text{a2}}, L_{\text{a3}} \in \text{Sparse}_{01379}$	Addition line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
W or $W^{-1} \in \mathbb{F}_{q^{12}}$	Residue witness	11	12
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	60C Evals, 8C Muls and 1C RLC		**69**
Q *(Amortized See* **??***)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	76	77

Table 6: Non-zero bit operation polynomial components

The residue witness W encapsulates the equivalence class relationship established by Novakovic and Eagen's final exponentiation elimination. The inverse of W is used for negative bits, this just changes the coefficients the equation remains the same.

4.4.3 Miller loop: Frobenius mapping Equation

The correction step finalizes the Miller loop by incorporating the Frobenius endomorphism corrections (lines **??** and **??** in Algorithm **??**). Again, we have two Sparse_{01379} lines for each pair, for π_p and $-\pi_p^2$ mappings, without any squaring of the accumulator. The verification equation becomes:

$$F(x) \cdot L_{1\pi}(x) \cdot L_{2\pi}(x) \cdot L_{3\pi}(x) \cdot L_{1\pi^2}(x) \cdot L_{2\pi^2}(x) \cdot L_{3\pi^2}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (4)$$

Polynomial	**Description**	**Degree**	**Coefficients**
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	11	12
$L_{1\pi}, L_{2\pi}, L_{3\pi} \in \text{Sparse}_{01379}$	π_p correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{1\pi^2}, L_{2\pi^2}, L_{3\pi^2} \in \text{Sparse}_{01379}$	$-\pi_p^2$ correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	48C Evals, 6C Muls and 1C RLC		**55**
Q *(Amortized See **??**)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	54	55

Table 7: Correction step polynomial components

4.4.4 Final Exponentiation Equation

The final exponentiation check is handled as described in [**?**]. The equation handles the cubic scale factor mutiplication, and **Frobenius component** $(q - q^2 + q^3)$ of the final exponentation (lines **??**, **??** and **??** in Algorithm **??**). The verification equation looks like:

$$F(x) \cdot c^{-q}(x) \cdot c^{q^2}(x) \cdot c^{-q^3}(x) \cdot W^w(x) = 1 + Q(x) \cdot P_{12}(x) \quad (5)$$

where $w \in \{0, 1, 2\}$ determines the cubic root of unity scaling factor, and the Frobenius mappings are computed using the residue witness c and its inverse and provided as input. Cubic root is sparse element with just 2 coefficients the degree ranging from 8 to 10 because of positioning of the coefficients. Our R is now 1 for a successful proof verification.

Polynomial	**Description**	**Degree**	**Coefficients**
$F \in \mathbb{F}_{q^{12}}$	Final Miller loop result	11	12
$c^{-q} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q$	11	12
$c^{q^2} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius q^2	11	12
$c^{-q^3} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q^3$	11	12
$W^w \in \mathbb{F}_{q^{12}}$	Cubic scale factor	10	2
$R = 1$	Unity (constant)	0	1
Total constraints	51C Evals, 4C Muls and 1C RLC		**56**
Q *(Amortized See **??**)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	43	44

Table 8: Final exponentiation polynomial components

With this we have all the polynomials we need for verifying entire pairing operation. Next we will look into high level overview of pairing verification via an interactive oracle proof.

4.5 Security

We analyze the security of our polynomial verification scheme.

4.5.1 Polynomial Identity Testing

A polynomial P of degree d over a field $\mathbb{F}_q$ has at most d roots in $\mathbb{F}_q$. Thus for a non-zero univariate polynomial evaluated at a random point $r \xleftarrow{\$} \mathbb{F}_q$, the probability of zero output is,

$$\Pr[P(r) = 0] \leq \frac{d}{|\mathbb{F}_q|}$$

4.5.2 Euclidean Division

Given polynomials A and B in $\mathbb{F}_q[x]$ with $B \neq 0$, there exist unique polynomials Q and R where $\deg(R) < \deg(B)$ such that:

$$A(x) = B(x) \cdot Q(x) + R(x)$$

4.5.3 Security Analysis

Our polynomials are of the form $P = A_1(x) \cdot A_2(x) \cdots A_n(x) - Q(x) \cdot P_{12}(x) - R(x)$ where $\deg(R) \leq 11$ and $\deg(P) < 2^8$. For verifying the computation of R with respect to P_{12}:

Soundness: The soundness error is $\delta_s = \Pr[P(r) = 0] \leq \frac{d}{|\mathbb{F}_q|}$ where $d = \deg(P) < 2^8$[2]. For the 254-bit BN254 field, $|\mathbb{F}_q| \approx 2^{254}$, so $\delta_s \leq 2^{-246}$.

Completeness: If the prover is honest and provides correct Q and R, then $P(x) = 0$ for all $x \in \mathbb{F}_q$. Thus, the verifier always accepts.

5 Full Pairing Proof with Quotients batching

Now that we have covered the core construction behind our contribution, let's take a holistic view of verification of a pairing computation with an interactive proof system. While the construction above works beautifully on it's own, we can further optimize the protocol by batching all polynomial verifications into a two round interactive proof.

5.1 Batching Polynomial Identity Testing

The whole process of polynomial verification can be batched into a single polynomial verification. Here's how a two round interactive proof system would look like,

The prover can commit all Rs for all n equations involved in pairing computation in the first interaction and receive a random challenge z. The challenge z can then be used to batch all the Qs into a single polynomial as a consecutive powers random linear combination like this,

[2]For k-pair pairings, the maximum degree is approximately $88 + 18(k-3)$, remaining well below 2^8 for most practical purposes. But can be increased to 2^{10} for many more pairs with negligible change in soundness.

$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i(z)$$

The verifier then prepares a new challenge c and checks,

$$\sum_{i=0}^{n-1} z^i \cdot A_i(c) \cdot B_i(c) \cdots X_i(c) = \sum_{i=0}^{n-1} z^i \cdot R_i(c) + Q_{\text{acc}}(c) \cdot P_{12}(c)$$

or,

$$\sum_{i=0}^{n-1} z^i \cdot [A_i(c) \cdot B_i(c) \cdots X_i(c) - R_i(c)] = Q_{\text{acc}}(c) \cdot P_{12}(c) \tag{6}$$

This reduces n polynomial evaluations of degree ranging from 38 to 76 (as described in Tables **??**, **??**, **??**, **??**) to a single polynomial of degree 76.

5.2 Prover's Algorithms

The prover computes quotients Q_i and remainders R_i for each polynomial equation described in Section **??**. Given input point pairs $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, the residue witness c as computed in [**?**], and internal parameter W (the 27th root of unity), the prover runs the pairing computation while simultaneously performing Euclidean division (Algorithm **??**) at each step to produce the verification polynomials. This process is detailed in Algorithm **??**. After receiving the challenge z from the verifier, the prover computes the random linear combination (Algorithm **??**) of all quotients Q_i to produce Q_{acc}. Prover then sends Q_{acc} and all remainders R_i to the verifier.

Algorithm 3 PolyMulDiv: Polynomial multiplication and Euclidean division

Input: $P_1, P_2, \ldots, P_n \in \mathbb{F}_q[x]$
Internal: Divisor $P_{12} \in \mathbb{F}_q[x]$
Output: $Q, R \in \mathbb{F}_q[x]$ where $\deg(R) < \deg(P_{12})$ and $\prod_{i=1}^{n} P_i = Q \cdot P_{12} + R$

$Q \leftarrow 0$, $R \leftarrow P_1$ ▷ Initialize remainder with first polynomial, Q as 0
for $i = 2$ to n **do** ▷ Start from second polynomial
    $R \leftarrow R \cdot P_i$ ▷ Multiply polynomials into R
end for
$d_P \leftarrow \deg(P_{12})$, $d_R \leftarrow \deg(R)$, $\ell_P \leftarrow \text{leading\_coeff}(P_{12})$
while $d_R \geq d_P$ **do** ▷ Euclidean division loop
    $t \leftarrow \text{leading\_coeff}(R)/\ell_P \cdot x^{d_R - d_P}$
    $Q \leftarrow Q + t$, $R \leftarrow R - t \cdot P_{12}$, $d_R \leftarrow \deg(R)$
end while
return (Q, R) ▷ Quotient and Remainder with $\deg(R) < \deg(P_{12})$

5.3 Verifier's Algorithms

The verifier receives point pairs $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, the residue witness c from [**?**], the internal parameter W (the 27th root of unity), computed remainders $\mathcal{R}$ for each polynomial equation described in Section **??** and Q_{acc} from the prover. The verifier

Algorithm 4 Random Linear Combination: Adjacent Powers

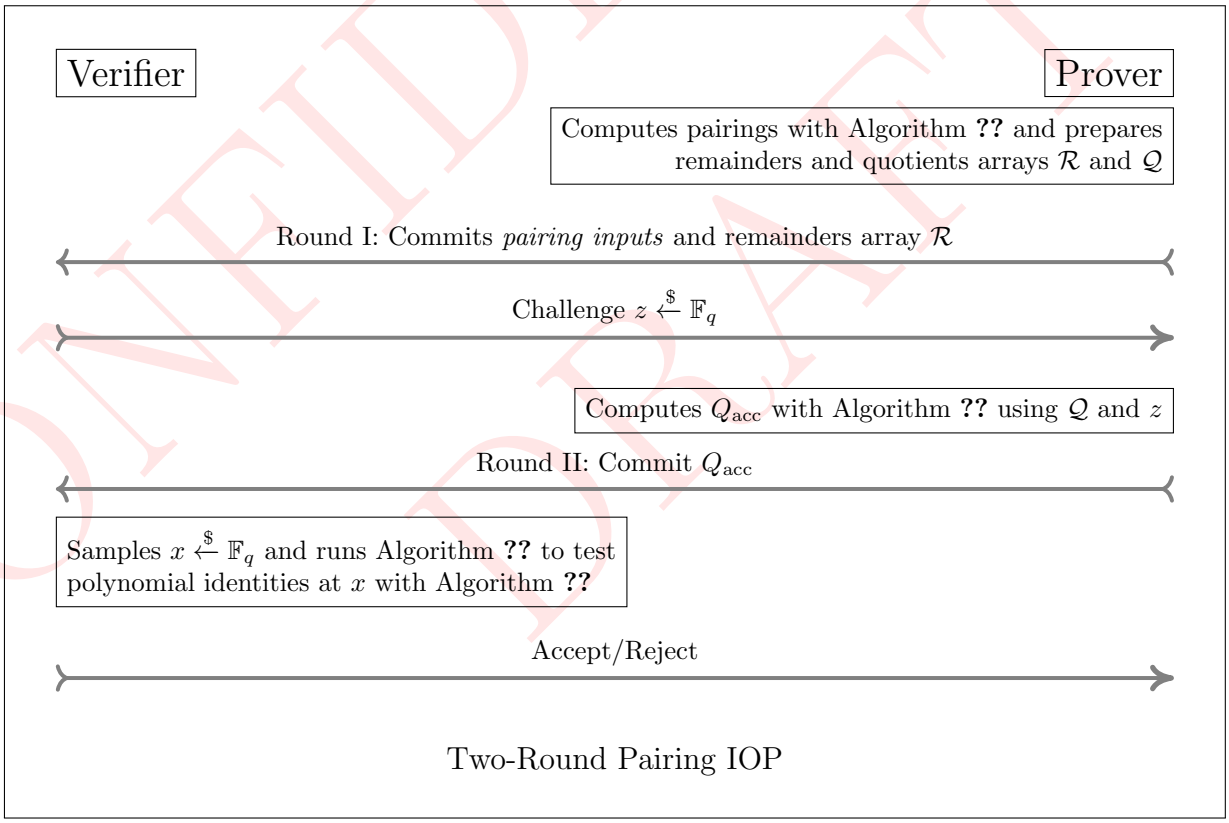
Input: $P_1, P_2, \ldots, P_n \in \mathbb{F}_q[x]$, $z \in \mathbb{F}_q$
Output: $\sum_{i=1}^{n} z^{i-1} P_i(z)$

1: $P_{\text{acc}} \leftarrow P_1$ ▷ Initialize P_{acc} with first polynomial
2: **for** $i = 2$ to n **do** ▷ Start from second polynomial
3: $\quad P_{\text{acc}} \leftarrow P_{\text{acc}} + z^{i-1} P_i(z)$
4: **end for**
5: **return** P_{acc}

retains the challenge c from the first round. Using these values, the verifier performs a pairing computation with polynomial evaluations (Algorithm **??**) at each step, as detailed in Algorithm **??**. After each evaluation, the verifier attempts to reconstruct RLC Q_{acc} the same way as the prover did, And checks the final equality described in Equation **??** on page **??**.

5.4 Interactive proof

Let's look into how the algorithms describe above come together to prove and verify a pairing computation. We will go through the two-round public-coin interactive proof system in this section and in the next section we will see how to convert it into a non-interactive proof using Fiat-Shamir transformation.



Algorithm 5 Pairing Prover: Preparing polynomials

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0,1,2\}$
Internal: $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: Arrays $\mathcal{Q}, \mathcal{R}$ of quotients and remainders
Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$

$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits
for $i = L - 2$ to 0 **do**
  $L_{\text{dbl}} \leftarrow l_{T,T}(P)$, $T \leftarrow [2]T$
  if $s_i = 0$ **then**
    $Q, R \leftarrow PolyMulDiv(f, f, L_{\text{dbl}})$ ▷ Algorithm **??**, add L_{*i} for each P_i, Q_i
  else if $s_i = 1$ **then**
    $L_{\text{add}} \leftarrow l_{T,Q}(P)$, $T \leftarrow T + Q$
    $Q, R \leftarrow PolyMulDiv(f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algorithm **??**, add L_{*i} for each P_i, Q_i
  else if $s_i = -1$ **then**
    $L_{\text{add}} \leftarrow l_{T,-Q}(P)$, $T \leftarrow T - Q$
    $Q, R \leftarrow PolyMulDiv(f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algorithm **??**, add L_{*i} for each P_i, Q_i
  end if
  $f \leftarrow R$, $\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$
end for

Miller loop Frobenius correction
$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$L_\pi \leftarrow l_{T,Q_1}(P)$
$T \leftarrow T + Q_1$
$L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
$Q, R \leftarrow PolyMulDiv(f, L_\pi, L_{-\pi^2})$ ▷ Algorithm **??**, add L_{*i} for each P_i, Q_i
$f \leftarrow R$, $\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$

Final exponentiation: Cubic scaling + Frobenius mappings
if $w = 0$ **then**
  $Q, R \leftarrow PolyMulDiv(f, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algorithm **??**
else if $w = 1$ **then**
  $Q, R \leftarrow PolyMulDiv(f, W, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algorithm **??**
else if $w = 2$ **then**
  $Q, R \leftarrow PolyMulDiv(f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algorithm **??**
end if
$\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$ ▷ Final $\mathcal{Q}$ and $\mathcal{R}$ accumulation, skip f

return $\mathcal{Q}, \mathcal{R}$

Algorithm 6 EvalPoly: Evaluate polynomial product minus remainder

Input: $R \in \mathbb{F}_q[x]$, $z \in \mathbb{F}_q$, $P_1, P_2, \ldots, P_n \in \mathbb{F}_q[x]$
Output: $e \in \mathbb{F}_q = \prod_{i=1}^{n} P_i(z) - R(z)$

1: $e \leftarrow P_1(z)$ ▷ Initialize evaluation with first polynomial
2: **for** $i = 2$ to n **do** ▷ Start from second polynomial
3: $\quad e \leftarrow e \cdot P_i(z)$ ▷ Multiply polynomial evaluations into e
4: **end for**
5: $e \leftarrow e - R(z)$ ▷ Subtract evaluation of R
6: **return** e

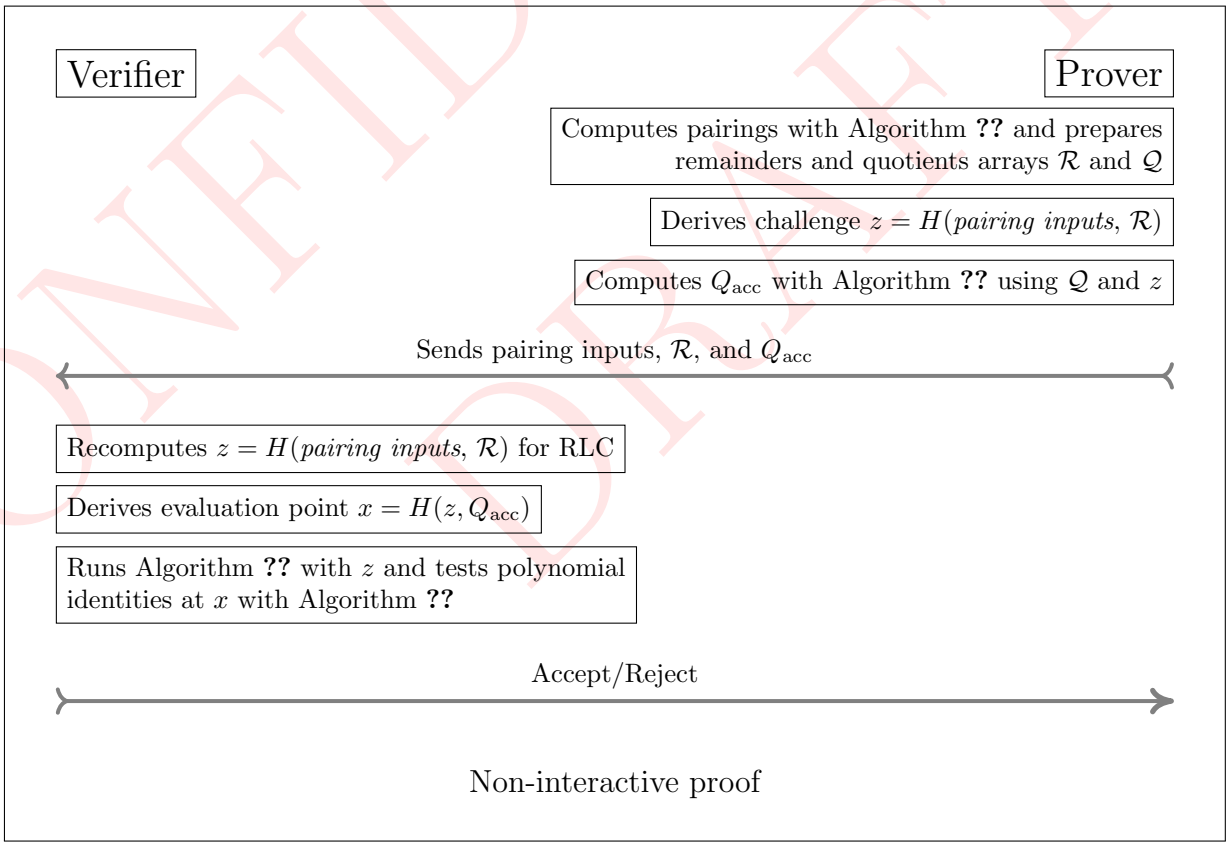
5.5 Fiat-Shamir Transformation

The Fiat-Shamir transformation [**?**] converts an interactive proof into a non-interactive one by replacing the verifier's random challenge with a hash of the protocol transcript.

This protocol, instantiated using the Fiat-Shamir transformation with hash function H modeled as a random oracle, achieves knowledge soundness of $\delta_s + 3(m^2+1)2^{-\lambda}$ [**?**], where δ_s is the soundness error for the interactive protocol (see Section **??**), and m is the number of hash queries performed (two for our protocol), and λ is the hash output length.

5.6 Non-interactive proof

Let's look at what the protocol looks like after applying Fiat-Shamir transformation.



Concrete Fiat-Shamir costs: In our Cairo implementation, each POSEIDON hash invocation costs approximately 25-30 Cairo steps. Our batching strategy reduces the

Algorithm 7 Pairing Verifier: Evaluating remainders

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0,1,2\}$, $\mathcal{R}$ array of remainders, $Q_{\text{acc}} \in \mathbb{F}_q[x]$

Internal: 27th root of unity, $W, W^2 \in \mathbb{F}_{q^{12}}$, first round challenge, $z \xleftarrow{\$} \mathbb{F}_q$

Output: $pairing(P,Q) \stackrel{?}{=} 1$

Write $s = 6t+2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1,0,1\}$ and $s_L = 1$

$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness

$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness

$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing

$x \xleftarrow{\$} \mathbb{F}_q$ ▷ Sample evaluation point from field

$e_{\text{acc}} \leftarrow 0$, $a_{\text{rlc}} \leftarrow 1$ ▷ Evaluation accumulator and random linear combination multiplier

Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits

for $i = L-2$ to 0 **do**

  $L_{\text{dbl}} \leftarrow l_{T,T}(P)$, $T \leftarrow [2]T$

  $R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover

  if $s_i = 0$ **then**

    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, f, L_{\text{dbl}})$ ▷ Algo **??**, add L_{*i} for each P_i, Q_i

  else if $s_i = 1$ **then**

    $L_{\text{add}} \leftarrow l_{T,Q}(P)$, $T \leftarrow T + Q$

    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo **??**, add L_{*i} for each P_i, Q_i

  else if $s_i = -1$ **then**

    $L_{\text{add}} \leftarrow l_{T,-Q}(P)$, $T \leftarrow T - Q$

    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo **??**, add L_{*i} for each P_i, Q_i

  end if

  $f \leftarrow R$, $a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

end for

Miller loop Frobenius correction

$R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover

$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$

$L_\pi \leftarrow l_{T,Q_1}(P)$

$T \leftarrow T + Q_1$

$L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$

$e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, L_\pi, L_{-\pi^2})$ ▷ Algo **??**, add L_{*i} for each P_i, Q_i

$f \leftarrow R$, $a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

Final exponentiation: Cubic scaling + Frobenius mappings

$R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover

if $w = 0$ **then**

  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo **??**

else if $w = 1$ **then**

  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, W, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo **??**

else if $w = 2$ **then**

  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo **??**

end if

return $Q_{\text{acc}}(x) \stackrel{?}{=} e_{\text{acc}} \wedge R \stackrel{?}{=} 1$ ▷ Polynomial identity and expected pairing output check

number of hash operations from $O(k \cdot n)$ to $O(k)$ where k is the Miller loop length and n is the number of pairings. For BN254 with $k = 66$ and $n = 3$, this yields the 77% reduction observed in Table **??**.

5.7 Security Analysis

We analyze the security of our random linear combination batching scheme against a malicious prover attempting to produce an incorrect remainder.

5.7.1 Attack Model

Consider a prover attempting to cheat by submitting an incorrect remainder $R'_j \neq R_j$ within the commitment array $\mathcal{R}$. The prover commits to $\mathcal{R}'$ containing R'_j, after which the challenge z is derived (via Fiat-Shamir) from this commitment or received from the verifier in interactive setting. We have Δ_j as:

$$\Delta_j = R'_j - R_j \neq 0, \text{ and } \deg(\Delta_j) \leq \deg(R_j)$$

5.7.2 Verifier's checks

For the verifier's check to pass with the incorrect remainder, the following equation must hold:

$$\sum_{i=0}^{s} z^i (A_i - R'_i) \stackrel{?}{=} Q_{\text{acc}} \cdot P_{12}$$

Substituting $R'_j = R_j + \Delta_j$ reveals an imbalance:

$$\sum_{i=0}^{s} z^i (A_i - R_i) - z^j \Delta_j = Q_{\text{acc}} \cdot P_{12}$$

5.7.3 Absorbing Δ in $\mathcal{R}$

Absorbing Δ_j into the remainders array $\mathcal{R}$ would require the knowledge of z before committing to $\mathcal{R}$. However, since z is derived from the commitment of $\mathcal{R}$ itself (in non-interactive setting) or received after committing to $\mathcal{R}$ (in interactive setting), this temporal ordering prevents the prover from adjusting $\mathcal{R}$ based on z, making it impossible to absorb Δ_j into another R_k in $\mathcal{R}$.

5.7.4 Absorbing Δ in Q_{acc}

Rearranging:

$$\sum_{i=0}^{s} z^i (A_i - R_i) = Q_{\text{acc}} \cdot P_{12} + z^j \Delta_j$$

For the proof to be accepted, the prover must find a polynomial Q'_{acc} satisfying:

$$Q'_{\text{acc}} \cdot P_{12} = Q_{\text{acc}} \cdot P_{12} + z^j \Delta_j$$

$$Q'_{\text{acc}} = Q_{\text{acc}} + \frac{z^j \Delta_j}{P_{12}}$$

Here P_{12} is irreducible polynomial with $\deg(P_{12}) = 12$, and $\deg(\Delta_j) \leq \deg(R_j) \leq 11$, so we have $\deg(\Delta_j) < \deg(P_{12})$ for all valid j. And since $\deg(\Delta_j) < \deg(P_{12})$, the division $z^j \Delta_j / P_{12}$ yields a rational function, not a polynomial. When you divide a polynomial of degree lesser than the divisor you get a rational function. So it is **information-theoretically impossible** to find the polynomial Q'_{acc}.

Thus, no polynomial $Q'_{\text{acc}} \in \mathbb{F}_q[x]$ exists that satisfies the verifier's equation with the incorrect remainder R'_j.

5.7.5 Soundness

As seen above, algebraic forgery is impossible. Therefore, security reduces to analysing the probability that a malicious prover succeeds through random chance.

Our random linear combination batching scheme (as described in Section **??**) effectively converts our univariate polynomial verifications into a single bivariate polynomial identity test:

$$P_{\text{final}}(z, x) = \sum_{i=0}^{n-1} z^i \cdot [\prod_j P_{i,j}(x) - R_i(x)] - Q_{\text{acc}}(x) \cdot P_{12}(x)$$

If a prover commits to incorrect remainder(s), P_{err} becomes a non-zero bivariate polynomial with:

- Degree in z: at most $n - 1$ (from the RLC structure)
- Degree in x: less than 2^8 (from the polynomial products)

By the Schwartz-Zippel lemma (Section **??**), the probability that P_{err} evaluates to zero at randomly chosen $(z, x) \in \mathbb{F}_q^2$ is bounded by:

$$\delta_s = \Pr[P_{\text{err}}(z, x) = 0] \leq \frac{\deg_z(P_{\text{err}}) + \deg_x(P_{\text{err}})}{|\mathbb{F}_q|} \leq \frac{n + 2^8}{|\mathbb{F}_q|}$$

Concrete parameters for BN254: With $n = 67$ equations (66 Miller loop steps plus 1 final exponentiation equation) and $|\mathbb{F}_q| \approx 2^{254}$:

$$\delta_s \leq \frac{67 + 256}{2^{254}} < \frac{2^9}{2^{254}} = 2^{-245}$$

Non-interactive proof via Fiat-Shamir: When transformed into a non-interactive proof with Fiat-Shamir under random oracle model, the knowledge soundness error becomes [**?**]:

$$\delta'_s = \delta_s + 3(m^2 + 1) \cdot 2^{-\lambda}$$

where $m = 2$ (hash queries) and $\lambda = 254$ (hash output length). Thus:

$$\delta'_s \leq 2^{-245} + 15 \cdot 2^{-254} < 2^{-244}$$

5.7.6 Completeness

For an honest prover executing Algorithm **??** with valid inputs:

1. **Polynomial construction**: At each operation, the prover computes following polynomials:
$$\prod_j P_{i,j} = Q_i \cdot P_{12} + R_i$$
2. **RLC formation**: Given challenge z, the prover forms:
$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$$
3. **Verifier's check**: The verifier evaluates at point x:
$$\begin{aligned}\text{LHS} &= \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x)\right] \\ &= \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \cdot P_{12}(x) \quad \text{(by construction)} \\ &= P_{12}(x) \cdot \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \\ &= P_{12}(x) \cdot Q_{\text{acc}}(x) = \text{RHS}\end{aligned}$$

Since the equality holds for any evaluation point x, the protocol satisfies perfect completeness.

6 Conclusion

6.0.1 Implementation Validation

Table **??** presents measurements from Garaga's production Cairo implementation on Starknet. For 3-pairing Miller loop verification:

Theoretical prediction (Section ??):

- Constraint reduction: 47% (76C → 40C per operation)
- Commitment reduction: 75% (48 → 12 $\mathbb{F}_p$ elements per operation)

Measured results:

- Field operations (MULMOD): 45% reduction (14,456 → 7,975)
- Hash operations (POSEIDON): 77% reduction (3,758 → 876)
- Random linear combinations: 80% reduction (329 → 66)

The close alignment validates our analysis. The hash reduction exceeds our prediction because batching eliminates not just intermediate commitments but also the associated Fiat-Shamir challenge derivations.

Table 9: Performance comparison: Cairo implementation on Starknet

Operation	**MULMOD**		**POSEIDON**		**Cycles**	
	Before	After	Before	After	Before	After
Miller Loop (BN254)						
n=1 pair	5,984	3,303	1,810	828	101,558	53,130
n=2 pairs	10,132	5,639	2,740	852	167,298	81,898
n=3 pairs	14,456	7,975	3,758	876	236,382	110,666
Improvement	**45%**		**77%**		**53%**	
Complete Pairing (BN254)						
n=1 pair	10,670	7,984	3,741	2,759	203,854	155,366
n=2 pairs	14,818	10,320	4,671	2,783	269,594	184,134
n=3 pairs	19,142	12,656	5,689	2,807	338,678	212,902
Improvement	**34%**		**51%**		**37%**	
Complete Pairing (BLS12_381)						
n=2 pairs	13,158	9,541	4,609	3,145	269,421	197,185
n=3 pairs	16,484	11,287	5,421	3,167	325,757	219,155
Improvement	**32%**		**42%**		**33%**	

MULMOD: Modular multiplications in $\mathbb{F}_p$; POSEIDON: Hash operations for Fiat-Shamir;
Cycles: Total Cairo VM execution cost. Improvements shown for n=3 (top) and average (bottom).

6.1 Performance

Batching operations in Miller loop steps yielded substantial performance improvements. Processing all operations within a Miller loop step as a single polynomial and performing Schwartz-Zippel verification of polynomial multiplications significantly reduced the overall computational requirements of the pairing verification process. Combined with residue witness technique for final exponentiation elimination [**?**], these optimizations enabled processing of ZK proofs on Starknet even before the modulo builtin [**?**] became available, which later greatly improved finite field arithmetic operations. These techniques were subsequently integrated into Garaga, which remains the most efficient pairing implementation by operation count on Starknet to date.